

ORDER MANAGEMENT SYSTEM

ONE8 Footwear E-commerce Database Management System

1. Database Design

1.1 Orders Table

Attribute	Data Type	Key Type / Constraint
Order_ID	NUMBER	Primary Key
Customer_ID	NUMBER	Foreign Key
Order_Date	DATE	NOT NULL
Total_Amount	NUMBER(10,2)	CHECK (> 0)
Order_Status	VARCHAR2(20)	NOT NULL
Payment_ID	NUMBER	Foreign Key

1.2 Order_Details Table

Attribute	Data Type	Key Type / Constraint
Order_Detail_ID	NUMBER	Primary Key
Order_ID	NUMBER	Foreign Key
Product_ID	NUMBER	Foreign Key
Quantity	NUMBER	CHECK (> 0)
Unit_Price	NUMBER(10,2)	CHECK (> 0)

2. Table Relationships

One Customer places many Orders (1 : M). Each Order contains one or more rows in Order_Details (1 : M), and each Order_Details row references exactly one Product (M : 1). Every Order is linked to one Payment record through Payment_ID.

Customer (Customer_ID) → Orders (Customer_ID) | Orders (Order_ID) → Order_Details (Order_ID) | Product (Product_ID) → Order_Details (Product_ID)

3. Table Creation

3.1 Create Orders Table

```
SQL> CREATE TABLE Orders (  
2     Order_ID NUMBER PRIMARY KEY,  
3     Customer_ID NUMBER,  
4     Order_Date DATE NOT NULL,  
5     Total_Amount NUMBER(10,2) CHECK (Total_Amount > 0),  
6     Order_Status VARCHAR2(20) NOT NULL,
```

```
7      Payment_ID NUMBER,  
8      FOREIGN KEY (Customer_ID) REFERENCES Customer(Customer_ID)  
9  );
```

Table created.

3.2 Create Order_Details Table

```
SQL> CREATE TABLE Order_Details (  
2      Order_Detail_ID NUMBER PRIMARY KEY,  
3      Order_ID NUMBER,  
4      Product_ID NUMBER,  
5      Quantity NUMBER CHECK (Quantity > 0),  
6      Unit_Price NUMBER(10,2) CHECK (Unit_Price > 0),  
7      FOREIGN KEY (Order_ID) REFERENCES Orders(Order_ID),  
8      FOREIGN KEY (Product_ID) REFERENCES Product(Product_ID)  
9  );
```

Table created.

4. Order Data Insertion

```
SQL>
SQL> INSERT INTO Orders VALUES
  2  (1, 201, TO_DATE('2026-09-10','YYYY-MM-DD'), 7998, 'Delivered');

1 row created.

SQL>
SQL> INSERT INTO Orders VALUES
  2  (2, 202, TO_DATE('2026-09-11','YYYY-MM-DD'), 4999, 'Shipped');

1 row created.

SQL>
SQL> INSERT INTO Orders VALUES
  2  (3, 203, TO_DATE('2026-09-12','YYYY-MM-DD'), 12497, 'Processing');

1 row created.

SQL>
SQL> INSERT INTO Orders VALUES
  2  (4, 204, TO_DATE('2026-09-13','YYYY-MM-DD'), 3499, 'Delivered');

1 row created.

SQL>
SQL> INSERT INTO Orders VALUES
  2  (5, 205, TO_DATE('2026-09-14','YYYY-MM-DD'), 8998, 'Shipped');

1 row created.
```

5. Order_Details Data Insertion

```
SQL> INSERT INTO Order_Details VALUES
  2  (1, 1, 1, 2, 3999);

1 row created.

SQL>
SQL> INSERT INTO Order_Details VALUES
  2  (2, 2, 2, 1, 4999);

1 row created.

SQL>
SQL> INSERT INTO Order_Details VALUES
  2  (3, 3, 3, 1, 5999);

1 row created.

SQL>
SQL> INSERT INTO Order_Details VALUES
```

```
2 (4, 4, 4, 1, 3499);

1 row created.

SQL>
SQL> INSERT INTO Order_Details VALUES
2 (5, 5, 5, 2, 4499);

1 row created.
```

6. Order Modification Operations

6.1 Modify Order Amount

```
SQL> UPDATE Orders
2 SET Total_Amount = 8499
3 WHERE Order_ID = 1;

1 row updated.

SQL> UPDATE Orders
2 SET Total_Amount = 5499
3 WHERE Order_ID = 2;

1 row updated.

SQL> SELECT Order_ID, Total_Amount
2 FROM Orders
3 WHERE Order_ID IN (1, 2);
```

ORDER_ID	TOTAL_AMOUNT
1	8499
2	5499

6.2 Modify Order Data

```
SQL> UPDATE Orders
  2 SET Total_Amount = 8998
  3 WHERE Order_ID = 1;

1 row updated.

SQL> UPDATE Orders
  2 SET Order_Status = 'Shipped'
  3 WHERE Order_ID = 2;

1 row updated.

SQL> SELECT Order_ID, Order_Date, Total_Amount, Order_Status
  2 FROM Orders
  3 WHERE Order_ID IN (1, 2);
```

ORDER_ID	ORDER_DAT	TOTAL_AMOUNT	ORDER_STATUS
1	10-SEP-26	8998	Delivered
2	11-SEP-26	5499	Shipped

7. Order_Details Modification

```
SQL> UPDATE Order_Details
  2 SET Quantity = 3
  3 WHERE Order_Detail_ID = 1;

1 row updated.

SQL> UPDATE Order_Details
  2 SET Unit_Price = 4299
  3 WHERE Order_Detail_ID = 2;

1 row updated.

SQL> SELECT Order_Detail_ID, Order_ID, Product_ID, Quantity, Unit_Price
  2 FROM Order_Details
  3 WHERE Order_Detail_ID IN (1, 2);
```

ORDER_DETAIL_ID	ORDER_ID	PRODUCT_ID	QUANTITY	UNIT_PRICE
1	1	1	3	3999
2	2	2	1	4299

8. Customer Order History Report

```
SQL> SELECT
  2 c.Customer_ID,
  3 c.FIRST_NAME || ' ' || c.LAST_NAME AS Customer_Name,
  4 o.Order_ID,
```

```

5      o.Order_Date,
6      p.Product_Name,
7      od.Quantity,
8      od.Unit_Price,
9      o.Total_Amount,
10     o.Order_Status
11 FROM Customer c
12 JOIN Orders o
13     ON c.Customer_ID = o.Customer_ID
14 JOIN Order_Details od
15     ON o.Order_ID = od.Order_ID
16 JOIN Product p
17     ON od.Product_ID = p.Product_ID
18 ORDER BY c.Customer_ID, o.Order_Date;

```

CUSTOMER_ID

CUSTOMER_NAME

ORDER_ID ORDER_DAT

PRODUCT_NAME

QUANTITY UNIT_PRICE TOTAL_AMOUNT ORDER_STATUS

201

Ayesha Khan

1 10-SEP-26

ONE8 Sprint Runner

9. Customer-wise Order Summary

```
SQL> SELECT
  2     c.Customer_ID,
  3     c.FIRST_NAME || ' ' || c.LAST_NAME AS Customer_Name,
  4     COUNT(o.Order_ID) AS Total_Orders,
  5     SUM(o.Total_Amount) AS Total_Amount
  6 FROM Customer c
  7 JOIN Orders o
  8   ON c.Customer_ID = o.Customer_ID
  9 GROUP BY
 10     c.Customer_ID,
 11     c.FIRST_NAME,
 12     c.LAST_NAME
 13 ORDER BY c.Customer_ID;
```

CUSTOMER_ID

CUSTOMER_NAME

TOTAL_ORDERS TOTAL_AMOUNT

201	
Ayesha Khan	
1	8998

202	
Rahul Verma	
1	5499